

Week 2 — Day 5: Query Performance, Storage Layout, and Data Formats

Practice Workbook

This workbook uses a fictional online retailer called **BritBooks**.

You will work with:

- Full table scans, indexes, and early filtering
- CSV, Parquet, Avro, ORC, and compression
- Partitioning, partition pruning, and small files
- Combined storage decisions for analytics

Practical exercises use SQLite and Python's standard library.
You do not need Spark, Hadoop, pandas, PyArrow, or a cloud account.

01
Query Performance

02
Storage Layout

03
Data Formats

04
Design Decisions

Workbook Roadmap

A visual guide to the four learning sections.

01

Query Performance Fundamentals

- Explain a full table scan.
- Understand when an index can help.
- Filter rows early and select only required columns.

02

Storage Layout and Partitioning

- Design a simple partition layout.
- Explain partition pruning.
- Recognise and reduce the small-files problem.

03

File Formats and Columnar Storage

- Compare CSV, Parquet, Avro, and ORC.
- Explain row-oriented and columnar storage.
- Understand basic compression and schema decisions.

04

Combined Design Decisions

- Combine query, partition, and format decisions.
- Design a simple analytics storage layout.
- Explain trade-offs using business requirements.

How to Use This Workbook

1. Read the scenario and requirements.
 2. Attempt the question before opening the answer.
 3. Run practical scripts independently.
 4. Compare your result with the collapsible solution.
 5. Change the sample data and try again.
-

01

Part 1 — Query Performance Fundamentals

WHAT YOU WILL LEARN

- Explain a full table scan.
- Understand when an index can help.
- Filter rows early and select only required columns.

Q. 1 — Full Table Scans

QUESTION

BritBooks has an `orders` table with one million rows.

A query searches for one customer but there is no index on `customer_id`.

Explain:

- What the database may need to read
- Why the query becomes slower as the table grows
- One way to improve the query

SHOW SUGGESTED ANSWER

The database may read every row to find matching customers. This is called a **full table scan**.

As the table grows, more rows must be checked, so the query normally takes longer.

An index on `customer_id` can help the database find matching rows without checking the entire table.

Q. 2 — Indexing Basics

QUESTION

BritBooks frequently runs this query:

SQL

```
SELECT order_id, amount_gbp
FROM orders
WHERE customer_id = 101;
```

Answer the following:

- Which column should be indexed?
- How can the index improve the query?
- What is one disadvantage of adding an index?

SHOW SUGGESTED ANSWER

- Create an index on `customer_id`.
- The index helps the database locate matching rows more directly.
- Indexes use additional storage and add work when rows are inserted, updated, or deleted.

Q. 3 — Practical: Observe a Scan and an Index

LAB

BritBooks wants to inspect how SQLite finds orders for one customer.

REQUIRED TABLE

Column	Data type
<code>order_id</code>	<code>INTEGER PRIMARY KEY</code>
<code>customer_id</code>	<code>INTEGER NOT NULL</code>
<code>amount_gbp</code>	<code>REAL NOT NULL</code>
<code>status</code>	<code>TEXT NOT NULL</code>

TASK

- Create `query_performance.db`.
- Create the `orders` table.
- Insert 5,000 sample orders.
- Use `EXPLAIN QUERY PLAN` before creating an index.
- Create an index on `customer_id`.
- Run `EXPLAIN QUERY PLAN` again.
- Print both query plans.

STARTER CODE

PYTHON

```
import sqlite3

connection = sqlite3.connect("query_performance.db")
cursor = connection.cursor()

# Create and populate the table
# Check the plan before the index
# Create the index
# Check the plan after the index

connection.close()
```

SHOW SOLUTION

PYTHON

```
import sqlite3

connection = sqlite3.connect("query_performance.db")
cursor = connection.cursor()

cursor.execute("DROP TABLE IF EXISTS orders")

cursor.execute("""
```

```
CREATE TABLE orders (  
    order_id INTEGER PRIMARY KEY,  
    customer_id INTEGER NOT NULL,  
    amount_gbp REAL NOT NULL,  
    status TEXT NOT NULL  
)  
""")  
  
orders = [  
    (  
        order_id,  
        (order_id % 500) + 1,  
        round(10 + (order_id % 90), 2),  
        "Dispatched" if order_id % 2 == 0 else "Confirmed"  
    )  
    for order_id in range(1, 5001)  
]  
  
cursor.executemany(  
    "INSERT INTO orders VALUES (?, ?, ?, ?)",  
    orders  
)  
connection.commit()  
  
query = ""  
SELECT order_id, amount_gbp  
FROM orders  
WHERE customer_id = 101  
""  
  
cursor.execute(f"EXPLAIN QUERY PLAN {query}")  
print("Before index:")  
for row in cursor.fetchall():  
    print(row)  
  
cursor.execute("""  
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id)  
""")  
connection.commit()  
  
cursor.execute(f"EXPLAIN QUERY PLAN {query}")  
print("\nAfter index:")  
for row in cursor.fetchall():  
    print(row)  
  
connection.close()
```

Expected Observation

Before the index, the plan should contain wording similar to:

TEXT

SCAN orders

After the index, it should contain wording similar to:

TEXT

```
SEARCH orders USING INDEX idx_orders_customer_id
```

i Exact SQLite output can vary slightly by version.

Q. 4 — Practical: Filter Early

LAB

BritBooks needs the total value of dispatched orders.

TASK

Create an SQLite script that compares these approaches:

Approach A

- Read every order.
- Filter `Dispatched` rows in Python.
- Calculate the total in Python.

Approach B

- Filter rows in SQL.
- Calculate the total using `SUM`.
- Return only the final value.

Print both totals and confirm that they match.

SHOW SOLUTION

PYTHON

```
import sqlite3

connection = sqlite3.connect("filter_early.db")
cursor = connection.cursor()

cursor.execute("DROP TABLE IF EXISTS orders")

cursor.execute("""
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    amount_gbp REAL NOT NULL,
```

```
        status TEXT NOT NULL
    )
    """
)

orders = [
    (1, 34.99, "Dispatched"),
    (2, 18.50, "Confirmed"),
    (3, 42.75, "Dispatched"),
    (4, 12.00, "Cancelled")
]

cursor.executemany(
    "INSERT INTO orders VALUES (?, ?, ?)",
    orders
)
connection.commit()

# Approach A: read everything and filter in Python
cursor.execute("SELECT * FROM orders")
all_orders = cursor.fetchall()

python_total = sum(
    amount
    for _, amount, status in all_orders
    if status == "Dispatched"
)

# Approach B: filter and aggregate in SQL
cursor.execute("""
SELECT SUM(amount_gbp)
FROM orders
WHERE status = 'Dispatched'
""")
sql_total = cursor.fetchone()[0]

print(f"Python total: £{python_total:.2f}")
print(f"SQL total: £{sql_total:.2f}")
print(f"Totals match: {python_total == sql_total}")

connection.close()
```

Expected Output

TEXT

```
Python total: £77.74
SQL total: £77.74
Totals match: True
```

Key Point

Filtering and aggregating in the database usually avoids transferring unnecessary rows and columns.

Q. 5 — MCQ: Full Table Scan

MCQ

What happens during a full table scan?

- A. The database reads only one indexed row.
- B. The database checks every row in the table.
- C. The database deletes unused rows.
- D. The database compresses the table.

[SHOW ANSWER](#)

Correct answer: B

Q. 6 — MCQ: Index Choice

MCQ

Which column is the best index candidate for this common query?

SQL

```
SELECT *  
FROM orders  
WHERE customer_id = 101;
```

- A. `customer_id`
- B. `amount_gbp`
- C. A column that is never queried
- D. Every column automatically

[SHOW ANSWER](#)

Correct answer: A — `customer_id`

Q. 7 — MCQ: Index Trade-off

MCQ

What is a common disadvantage of an index?

- A. It prevents all queries.
- B. It uses storage and adds write overhead.
- C. It removes primary keys.
- D. It converts rows into files.

SHOW ANSWER**Correct answer: B**

Q. 8 — MCQ: Filtering Early

MCQ

Which query transfers the least unnecessary data?

- A. `SELECT * FROM orders;`
- B. `SELECT * FROM orders;` followed by Python filtering
- C. `SELECT order_id FROM orders WHERE status = 'Dispatched';`
- D. `SELECT * FROM customers, orders;`

SHOW ANSWER**Correct answer: C**

It filters rows and selects only the required column.

Q. 9 — MCQ: Low-Cardinality Columns

MCQ

An `order_status` column contains only `Confirmed`, `Dispatched`, and `Cancelled`.

Why might an index on this column provide limited improvement for some queries?

- A. The column has very few distinct values.
- B. Text columns can never be indexed.
- C. Indexes work only on dates.
- D. Status values are stored in pounds.

SHOW ANSWER**Correct answer: A**

A filter may still match a large part of the table.

02

Part 2 — Storage Layout and Partitioning

WHAT YOU WILL LEARN

- Design a simple partition layout.
- Explain partition pruning.
- Recognise and reduce the small-files problem.

Q. 10 — Partitioning Fundamentals

QUESTION

BritBooks stores three years of order data.

Most reports filter by order date.

Explain:

- A suitable partition key
- A possible folder layout
- How partition pruning reduces the amount of data read

SHOW SUGGESTED ANSWER

A suitable key is the order date, usually split into **year** and **month**.

TEXT

```
/orders/year=2026/month=01/  
/orders/year=2026/month=02/
```

A query for February 2026 can read only the matching folder instead of scanning every year and month.

Q. 11 — Design a Partition Scheme

SCENARIO

BritBooks has:

- 100 million orders per year
- Reports that normally filter by month
- Very few reports for a single hour

QUESTION

Choose one layout and justify your decision:

Option A

TEXT

```
/orders/year=2026/month=07/
```

Option B

TEXT

```
/orders/year=2026/month=07/day=17/hour=10/
```

SHOW SUGGESTED ANSWER

Option A is more suitable.

Monthly filtering matches the normal query pattern. Hourly partitions would create many directories and could produce very small partitions.

Q. 12 — The Small-Files Problem

QUESTION

A daily partition contains 10,000 files, each approximately 50 KB.

Explain:

- Why opening many small files can be inefficient
- What compaction means
- When a compaction job could run

SHOW SUGGESTED ANSWER

Each file has opening and metadata overhead. Thousands of files can therefore take longer to list, open, and schedule than a smaller number of larger files.

Compaction combines many small files into fewer larger files.

BritBooks could run compaction after the daily load or during a quieter period.

Q. 13 — Practical: Combine Small CSV Files

LAB

BritBooks receives three small order files for one day.

INPUT FILES

TEXT

```
orders_1.csv
orders_2.csv
orders_3.csv
```

Each file contains:

TEXT

```
order_id,customer_name,amount_gbp
```

TASK

- Create a folder named `small_order_files`.
- Create the three CSV files with one order in each.
- Combine them into `orders_2026-07-17.csv`.
- Write the header only once.
- Print the number of combined rows.

SHOW SOLUTION

PYTHON

```
from pathlib import Path
import csv

input_folder = Path("small_order_files")
input_folder.mkdir(exist_ok=True)

files_and_rows = {
    "orders_1.csv": [1001, "Amelia Clarke", 34.99],
    "orders_2.csv": [1002, "Oliver Bennett", 18.50],
    "orders_3.csv": [1003, "Aisha Khan", 42.75]
}

header = ["order_id", "customer_name", "amount_gbp"]

for file_name, row in files_and_rows.items():
    with (input_folder / file_name).open(
        "w",
        newline="",
        encoding="utf-8"
    ) as file:
        writer = csv.writer(file)
```

```
        writer.writerow(header)
        writer.writerow(row)

output_file = Path("orders_2026-07-17.csv")
combined_rows = []

for input_file in sorted(input_folder.glob("orders_*.csv")):
    with input_file.open(
        "r",
        newline="",
        encoding="utf-8"
    ) as file:
        reader = csv.DictReader(file)
        combined_rows.extend(reader)

with output_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.DictWriter(file, fieldnames=header)
    writer.writeheader()
    writer.writerows(combined_rows)

print(f"Combined rows: {len(combined_rows)}")
print(f"Output file: {output_file}")
```

Expected Output

TEXT

```
Combined rows: 3
Output file: orders_2026-07-17.csv
```

Key Point

Real data platforms perform the same general operation at a larger scale and often write the result in Parquet or ORC.

Q. 14 — MCQ: Partition Key

MCQ

Which is the best partition key when most queries filter orders by month?

- A. Customer name
- B. Order month
- C. Product description
- D. Order amount rounded to the nearest pound

SHOW ANSWER**Correct answer: B — Order month**

Q. 15 — MCQ: Partition Pruning

MCQ

What does partition pruning do?

- A. Deletes old partitions automatically
- B. Reads only partitions that match the filter
- C. Creates an index on every column
- D. Converts CSV into JSON

SHOW ANSWER**Correct answer: B**

Q. 16 — MCQ: Over-Partitioning

MCQ

What is a possible result of partitioning data too finely?

- A. Too many small folders and files
- B. All rows become duplicates
- C. SQL stops supporting filters
- D. Compression becomes impossible

SHOW ANSWER**Correct answer: A**

Q. 17 — MCQ: Small Files

MCQ

Why are thousands of tiny files often inefficient?

- A. They cannot contain numbers.
- B. Each file adds metadata and opening overhead.
- C. They always delete their schemas.
- D. Small files cannot be stored in folders.

SHOW ANSWER**Correct answer: B**

Q. 18 — MCQ: Compaction

MCQ

What is compaction?

- A. Splitting one file into thousands of files
- B. Combining small files into fewer larger files
- C. Removing all partition columns
- D. Replacing SQL with Python

SHOW ANSWER**Correct answer: B**

03

Part 3 — File Formats and Columnar Storage

WHAT YOU WILL LEARN

- Compare CSV, Parquet, Avro, and ORC.
- Explain row-oriented and columnar storage.
- Understand basic compression and schema decisions.

Q. 19 — Compare Common File Formats

QUESTION

Complete the table using short phrases.

Format	Text or binary?	Row or column oriented?	Common use
CSV			
Parquet			
Avro			
ORC			

SHOW SUGGESTED ANSWER

Format	Text or binary?	Row or column oriented?	Common use
CSV	Text	Row-like records	Simple exchange and manual inspection
Parquet	Binary	Columnar	Data lake and analytics workloads
Avro	Binary	Row-oriented	Data exchange and streaming events
ORC	Binary	Columnar	Hive and Hadoop analytics

Q. 20 — Row and Columnar Storage

QUESTION

BritBooks has an analytics file with 80 columns.

A report needs only:

- `order_date`
- `product_id`
- `amount_gbp`

Explain why a columnar format may read less data than CSV.

SHOW SUGGESTED ANSWER

CSV stores complete rows, so a reader normally processes data from every column in each row.

A columnar format stores values from the same column together. An analytics engine can therefore read only the three required columns.

Q. 21 — Choose a File Format

QUESTION

Choose the most suitable option and give one reason.

Requirement	Choose from
A small file that a non-technical user can open easily	CSV / Parquet
Large analytical data where queries select a few columns	CSV / Parquet
Streaming events with an explicit schema	Avro / CSV
Large Hive analytics tables	ORC / CSV

SHOW SUGGESTED ANSWER

Requirement	Choice	Reason
Easy manual opening	CSV	It is plain text and widely supported.
Large analytical data	Parquet	It is columnar and supports compression.
Streaming events with schema	Avro	It stores structured records with a defined schema.
Hive analytics tables	ORC	It is a columnar format designed for large analytical workloads.

Q. 22 — Practical: Create an Avro-Style Schema File

LAB

BritBooks sends order events between systems.

REQUIRED FIELDS

Field	Type
<code>order_id</code>	string
<code>customer_id</code>	string
<code>amount_gbp</code>	double
<code>order_status</code>	string
<code>coupon_code</code>	null or string, default <code>null</code>

TASK

- Create the schema as a Python dictionary.
- Write it to `order_event_schema.json`.
- Use indentation to keep the JSON readable.
- Read the file again.
- Print the field names.

SHOW SOLUTION

PYTHON

```
import json

schema = {
    "type": "record",
    "name": "OrderEvent",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "customer_id", "type": "string"},
        {"name": "amount_gbp", "type": "double"},
        {"name": "order_status", "type": "string"},
        {
            "name": "coupon_code",
            "type": ["null", "string"],
            "default": None
        }
    ]
}

with open(
    "order_event_schema.json",
    "w",
    encoding="utf-8"
) as file:
    json.dump(schema, file, indent=2)

with open(
    "order_event_schema.json",
    "r",
```

```
encoding="utf-8"
) as file:
    loaded_schema = json.load(file)

    field_names = [
        field["name"]
        for field in loaded_schema["fields"]
    ]

    print(field_names)
```

Expected Output

```
TEXT

['order_id', 'customer_id', 'amount_gbp', 'order_status', 'coupon_code']
```

Key Point

The optional field has a default value, which helps older records remain readable when a new field is introduced.

Q. 23 — MCQ: CSV

MCQ

Which statement about CSV is correct?

- A. It always stores a strict schema.
- B. It is a human-readable text format.
- C. It is a columnar binary format.
- D. It automatically supports nested objects.

SHOW ANSWER

Correct answer: B

Q. 24 — MCQ: Parquet

MCQ

Why is Parquet commonly used for analytics?

- A. It stores every value as plain text.
- B. It is columnar and can read selected columns.
- C. It supports only one data type.
- D. It cannot be compressed.

SHOW ANSWER**Correct answer: B**

Q. 25 — MCQ: Avro

MCQ

Which use case best matches Avro?

- A. A simple spreadsheet opened manually
- B. Structured event exchange with schema evolution
- C. Image storage
- D. Graph traversal

SHOW ANSWER**Correct answer: B**

Q. 26 — MCQ: ORC

MCQ

ORC is mainly associated with:

- A. Columnar analytics in the Hadoop and Hive ecosystem
- B. Storing passwords
- C. Editing source code
- D. Relational transactions

SHOW ANSWER

Correct answer: A

Q. 27 — MCQ: Compression

MCQ

What is the main benefit of compression?

- A. It always removes CPU usage.
- B. It reduces storage size and data read from disk.
- C. It removes the need for a schema.
- D. It converts columns into rows.

SHOW ANSWER

Correct answer: B

Compression saves storage and input/output, although decompression uses some CPU.

04

Part 4 — Combined Design Decisions

WHAT YOU WILL LEARN

- Combine query, partition, and format decisions.
- Design a simple analytics storage layout.
- Explain trade-offs using business requirements.

Q. 28 — Identify the Requirements

SCENARIO

BritBooks needs a daily sales report.

The report:

- Filters by order month
- Groups by product
- Reads only order date, product ID, quantity, and amount
- Uses data stored for five years

QUESTION

Identify:

- The best partition field
- A suitable analytical file format
- The columns the query should select
- One reason to avoid `SELECT *`

SHOW SUGGESTED ANSWER

- Partition using order date, such as year and month.
- Use Parquet or ORC for analytical storage.
- Select only order date, product ID, quantity, and amount.

- `SELECT *` reads unnecessary columns and increases input/output.

Q. 29 — Create a Storage Decision Table

SCENARIO

BritBooks has these datasets:

- Raw order exports
- Cleaned analytical orders
- Streaming click events
- Monthly sales summaries

TASK

Create a table with:

Dataset	File format	Partition key	Reason
---------	-------------	---------------	--------

Use CSV, Parquet, Avro, or ORC where appropriate.

SHOW SUGGESTED ANSWER

Dataset	File format	Partition key	Reason
Raw order exports	CSV	Ingestion date	Keeps the original portable source files
Cleaned analytical orders	Parquet	Order year and month	Supports columnar analytics and pruning
Streaming click events	Avro	Event date and hour	Supports structured event records and schema evolution
Monthly sales summaries	Parquet or ORC	Summary year and month	Supports fast analytical reads

Q. 30 — Practical: Create a Partitioned Folder Layout

LAB

BritBooks wants to organise orders by year and month.

DATA

TEXT

```
1001, 2026-06-28, £34.99
1002, 2026-07-02, £18.50
1003, 2026-07-17, £42.75
```

TASK

- Create an `orders_lake` folder.
- Create `year=YYYY/month=MM` folders from each date.
- Write each order to an `orders.csv` file in the correct partition.
- Include the CSV header.
- Print the created file paths.

SHOW SOLUTION

PYTHON

```
from pathlib import Path
from datetime import datetime
import csv

orders = [
    {
        "order_id": 1001,
        "order_date": "2026-06-28",
        "amount_gbp": 34.99
    },
    {
        "order_id": 1002,
        "order_date": "2026-07-02",
        "amount_gbp": 18.50
    },
    {
        "order_id": 1003,
        "order_date": "2026-07-17",
        "amount_gbp": 42.75
    }
]

base_folder = Path("orders_lake")
header = ["order_id", "order_date", "amount_gbp"]

for order in orders:
    order_date = datetime.strptime(
        order["order_date"],
        "%Y-%m-%d"
    )
```

```
partition_folder = (
    base_folder
    / f"year={order_date.year}"
    / f"month={order_date.month:02d}"
)
partition_folder.mkdir(parents=True, exist_ok=True)

output_file = partition_folder / "orders.csv"
file_exists = output_file.exists()

with output_file.open(
    "a",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.DictWriter(file, fieldnames=header)

    if not file_exists:
        writer.writeheader()

    writer.writerow(order)

for file_path in sorted(base_folder.rglob("orders.csv")):
    print(file_path)
```

Expected Folder Layout

```
TEXT

orders_lake/
├─ year=2026/
│   ├── month=06/
│   │   └─ orders.csv
│   └─ month=07/
│       └─ orders.csv
```

Key Point

Queries filtering on year and month can avoid reading unrelated partitions.

Q. 31 — Review an Analytics Design

PROPOSED DESIGN

BritBooks stores:

- Five years of orders in one uncompressed CSV file
- 100 columns in every row
- Reports that normally use four columns

- No date partitions
- One daily process that appends thousands of tiny files

QUESTION

Recommend four improvements.

SHOW SUGGESTED ANSWER

- Convert cleaned analytics data to Parquet or ORC.
- Partition data by a commonly filtered date field.
- Select only required columns in queries.
- Compact the daily small files into fewer larger files.
- Use compression appropriate for the chosen analytical format.

Any four valid improvements are acceptable.

Q. 32 — MCQ: Analytics Format

MCQ

Which format is the strongest default choice for large analytical order data?

- A. Parquet
- B. Plain text without separators
- C. Image files
- D. HTML

SHOW ANSWER

Correct answer: A — Parquet

Q. 33 — MCQ: Partition Filter

MCQ

Which filter is most likely to support pruning for this layout?

TEXT

```
/orders/year=2026/month=07/
```

- A. `WHERE customer_name = 'Amelia Clarke'`
- B. `WHERE year = 2026 AND month = 7`
- C. `WHERE amount_gbp > 20`
- D. `WHERE status = 'Confirmed'`

SHOW ANSWER

Correct answer: B

Q. 34 — MCQ: Projection

MCQ

What does projection mean in query optimisation?

- A. Selecting only required columns
- B. Deleting old rows
- C. Creating additional small files
- D. Sorting every table

SHOW ANSWER

Correct answer: A

Q. 35 — MCQ: Streaming Events

MCQ

Which format is a suitable choice for structured streaming event records?

- A. Avro
- B. Bitmap image
- C. PDF
- D. Plain HTML

SHOW ANSWER

Correct answer: A — Avro

Q. 36 — MCQ: Combined Optimisation

MCQ

Which combination is most suitable for monthly sales analytics?

- A. One large CSV, `SELECT *`, and no date filter
- B. Parquet, date partitions, early filtering, and required columns only
- C. Thousands of tiny text files and no compression
- D. Graph storage for every sales total

SHOW ANSWER

Correct answer: B



Final Recap

Before finishing, confirm that you can:

- ☐ Explain a full table scan.
- ☐ Describe how an index can improve a query.
- ☐ Explain why indexes also have a cost.
- ☐ Filter rows early and select required columns.
- ☐ Design a basic date partition layout.
- ☐ Explain partition pruning.
- ☐ Describe the small-files problem and compaction.
- ☐ Compare CSV, Parquet, Avro, and ORC.
- ☐ Explain why columnar formats help analytics.
- ☐ Choose a format and layout from business requirements.

Practice Reminder

- Use small datasets while learning.
- Query timings can differ between computers.
- Focus on query plans and data-access patterns rather than exact timings.
- Do not add partitions or indexes without considering how the data is queried.